

Mise en situation professionnelle

Développement full stack mobile — Flutter

Filière : Master BIHAR

Module : Développement full stack mobile (Flutter & Dart 3)

Nature de l'épreuve : projet individuel — mise en situation professionnelle

Durée : 14 heures (2 journées)

Session : 2025-2026

Livrables : dossier de conception · application Flutter + dossier technique

Document remis à l'étudiant·e. À conserver pendant toute la durée de l'épreuve.

Sommaire

1. Présentation de l'épreuve
 2. Contexte professionnel
 3. Expression du besoin
 4. Livrables attendus
 5. Évaluation
 6. Recommandations et règles
- Annexe A — Checklist de conformité du rendu
- Annexe B — Correspondance compétences / formation
- Annexe C — Structure du JSON fourni

1. Présentation de l'épreuve

Cette épreuve terminale valide les compétences acquises pendant le module de développement full stack mobile. Elle prend la forme d'une mise en situation professionnelle. A partir d'un besoin client réaliste et d'un jeu de données fourni, vous concevez et développez, seule, une application mobile Flutter fonctionnelle sur iOS et Android, puis vous en documentez la conception et la réalisation.

Objectifs

- Analyser un besoin métier et le traduire en une conception logicielle argumentée.
- Concevoir et développer une application mobile Flutter structurée et maintenable.
- Consommer une source de données (JSON) et persister des modifications en local.
- Garantir la qualité : gestion d'erreurs, états de chargement,
- Documenter et présenter son travail de façon professionnelle.

Modalités

- **Durée** : 14 heures
- **Travail individuel**. L'ensemble des livrables et produit est personnel
- **Plateformes cibles** : iOS et Android (émulateurs/simulateurs acceptés pour la démonstration).
- **Technologie imposée** : Flutter (Dart 3), conformément au contenu de la formation.
- **Source de données** : un fichier JSON fourni par l'équipe pédagogique (voir Annexe C). Aucun backend à mettre en place.
- **Ressources autorisées** : documentation officielle, packages open-source (à citer), notes de cours et TP.
- **Remise** : dépôt Git + documents PDF (voir section 4).

2. Contexte professionnel

Le client — AgriVista. AgriVista est une entreprise corse qui installe et entretient des stations de capteurs connectés (humidité, température, ensoleillement) pour des exploitations agricoles et viticoles. Ses techniciens interviennent sur le terrain, souvent dans des zones à couverture réseau faible ou intermittente.

Votre rôle. Vous êtes recruté-e comme développeur-se mobile junior. La direction technique vous confie la réalisation de la première version de l'application interne « AgriVista Field », destinée aux techniciens de terrain pour gérer leurs interventions de maintenance.

La problématique métier. Aujourd'hui, les techniciens gèrent leurs interventions sur papier. Pour cette première version, l'application charge la liste des interventions depuis un fichier de données fourni (JSON), permet de les consulter et d'en faire évoluer le statut depuis le terrain. Les modifications sont enregistrées localement sur l'appareil et restent

disponibles après redémarrage — la synchronisation avec un serveur central est prévue pour une version ultérieure, hors périmètre.

En résumé

Vous devez livrer une application mobile de gestion d'interventions terrain qui lit ses données depuis un JSON fourni, permet d'en modifier le statut avec persistance locale, le tout structuré, testé et documenté (dossier de conception + dossier technique).

3. Expression du besoin

3.1 Fonctionnalités imposées (périmètre minimum)

L'application doit impérativement couvrir les fonctionnalités suivantes. Elles constituent le socle évalué.

Fonctionnalité	Description	Priorité
Chargement des données	Lire la liste des interventions depuis le JSON fourni et la désérialiser.	Haute
Liste des interventions	Afficher les interventions, avec filtres (statut, priorité) et recherche.	Haute
Détail d'une intervention	Station concernée, localisation, description, statut et historique.	Haute
Mise à jour du statut	Passage planifiée → en cours → terminée depuis l'application.	Haute
Persistance locale	Les statuts modifiés sont enregistrés localement et survivent au redémarrage.	Haute
États de chargement	Affichage des états chargement / succès / erreur lors de la lecture des données.	Haute
Thème & profil	Thème Material 3 cohérent et un écran de profil simple.	Moyenne

3.2 Fonctionnalités optionnelles (valorisées)

Une fois le périmètre minimum atteint, ces extensions permettent de se distinguer. Elles ne compensent pas un socle incomplet.

- Écran de connexion simulé (sans backend) et gestion d'une session locale.
- Thème sombre / clair et adaptation tablette (mise en page responsive).
- Rechargement des données (pull-to-refresh) et tri des interventions.
- Ajout d'une photo locale et d'une note à une intervention.
- Carte de localisation ou tableau de bord de synthèse (compteurs par statut).
- Animations soignées, internationalisation,

3.3 Contraintes techniques imposées

Ces contraintes traduisent directement les notions travaillées en formation et sont évaluées.

- **Framework** : Flutter (Dart 3), application mobile iOS et Android.
- **Architecture** : découpage en couches Data / Domain / Presentation (Clean Architecture).
- **Gestion d'état** : Riverpod (ou BLoC si le choix est justifié dans le dossier de conception).
- **Source de données** : le JSON fourni, chargé en lecture seule via un package HTTP (Dio ou http) s'il est distant, ou depuis les assets s'il est local. Le JSON ne doit pas être modifié.
- **Sérialisation** : conversion du JSON en objets typés (Freezed + json_serializable recommandés).
- **Persistance locale** : enregistrement des statuts modifiés dans une base locale (Isar ou Hive) ou un stockage clé-valeur adapté.
- **Réseau & données** : gestion explicite des états (chargement, succès, erreur) et erreurs typées.
- **Versonnement** : dépôt Git avec un historique de commits régulier et significatif.

4. Livrables attendus

Deux livrables sont attendus. Ils sont complémentaires : la conception précède et justifie la réalisation.

4.1 Livrable 1 — Dossier de conception

Document (PDF, format concis d'environ 6 à 10 pages) présentant la démarche de conception. Il doit contenir au minimum :

- La reformulation du besoin et le périmètre retenu (ce qui est inclus et exclu).
- Les cas d'usage / user stories principaux, avec les acteurs et les scénarios.
- Les maquettes ou wireframes des écrans clés et le parcours utilisateur.
- Le modèle de données : entités du domaine, en cohérence avec la structure du JSON fourni (Annexe C).
- L'architecture logicielle : schéma en couches, découpage des responsabilités, choix de gestion d'état — le tout justifié.
- Les choix techniques argumentés (lecture du JSON, persistance locale, packages majeurs) et les alternatives écartées.
- La stratégie de persistance locale des modifications de statut.

4.2 Livrable 2 — Application développée et dossier technique

a) L'application. Le code source complet, compilable, versionné sur un dépôt Git partagé avec l'équipe pédagogique. L'application doit se lancer sur iOS et sur Android et couvrir le périmètre minimum. Un fichier README décrit l'installation, la configuration (emplacement du JSON de données) et le lancement.

b) Le dossier technique. Document (PDF, ou README étendu) décrivant la réalisation. Il doit contenir au minimum :

- La pile technique et les versions utilisées.
- L'arborescence du projet et le rôle de chaque couche.
- La gestion d'état retenue et son fonctionnement.
- Le chargement du JSON, sa désérialisation et la gestion des états (chargement / succès / erreur).
- La stratégie de persistance locale des statuts modifiés.
- La gestion des erreurs.
- Les difficultés rencontrées, les solutions apportées, les limites et pistes d'amélioration (dont la future synchronisation serveur).

4.3 Format et remise

- **Dépôt Git** : lien vers un dépôt (privé) partagé avec l'équipe pédagogique, plus une archive du code à la date de remise.
- **Documents** : dossier de conception et dossier technique au format PDF.
- **Emplacement** : Livrables étudiants sur Whaller

- **Nommage des fichiers** : selon la convention de l'école, en identifiant clairement l'auteur·e (ex. Nom-Prenom).
- **Date limite** : 06/09 à 23h59

5. Évaluation

L'évaluation porte sur l'ensemble des livrables. Le barème ci-dessous est indicatif et peut être ajusté par l'équipe pédagogique.

Critère	Ce qui est évalué	Poids
Dossier de conception	Analyse du besoin, maquettes, modèle de données, architecture justifiée.	25 %
Application — fonctionnel	Couverture du périmètre minimum, fonctionnement sur iOS et Android.	25 %
Architecture & qualité du code	Découpage en couches, lisibilité, gestion d'état, gestion d'erreurs.	25 %
Données & persistance	Lecture du JSON, sérialisation, états de chargement, persistance locale.	20 %
Dossier technique & documentation	Clarté, complétude, README et instructions de build.	5 %

6. Recommandations et règles

Intégrité et bonnes pratiques

- Le projet est strictement individuel ; tout code réutilisé (extraits, packages) doit être cité.
- Le JSON fourni sert de source de données de référence et ne doit pas être modifié.
- Privilégiez un socle fonctionnel et propre avant d'ajouter des fonctionnalités optionnelles.
- Testez systématiquement sur les deux plateformes (iOS et Android) avant la remise.

Conseils pratiques :

- Commencez par la conception : un modèle de données clair fait gagner un temps considérable.
- Committez régulièrement, avec des messages explicites : l'historique fait partie de l'évaluation.

Annexe A — Checklist de conformité du rendu

À vérifier avant la remise. Chaque point non satisfait pénalise l'évaluation.

- ☐ Le dossier de conception (PDF) est complet et remis.
- ☐ Le dépôt Git est partagé avec l'équipe pédagogique et compile sans erreur.
- ☐ L'application se lance sur iOS et sur Android.
- ☐ Le périmètre minimum (section 3.1) est couvert.
- ☐ L'architecture est découpée en couches Data / Domain / Presentation.
- ☐ La gestion d'état utilise Riverpod (ou BLoC justifié).
- ☐ Les données sont chargées depuis le JSON fourni et désérialisées correctement.
- ☐ Les états de chargement, succès et erreur sont gérés.
- ☐ La modification du statut est persistée localement et survit au redémarrage.
- ☐ Le dossier technique (PDF ou README étendu) est complet.

Annexe B — Correspondance compétences / formation

Ce projet mobilise l'essentiel des cinq journées du module. Certaines notions vues en formation (backend temps réel, authentification distante, CI/CD) sont hors périmètre ou en bonus.

Jour	Notions	Où elles servent dans le projet
Jour 1	Dart 3, POO, async, widgets, layouts	Modèles du domaine, écrans et mise en page
Jour 2	Theming, animations, responsive, Riverpod	UI, thème, liste filtrable, gestion d'état
Jour 3	Clean Architecture, DI, persistance locale	Découpage en couches, statut persisté en local
Jour 4	Client HTTP, sérialisation JSON, états réseau	Lecture du JSON fourni, parsing, états de chargement

Annexe C — Structure du JSON fourni

L'équipe pédagogique fournit un fichier JSON de ce format. Les valeurs de statut sont : planifiée, en_cours, terminée ; les priorités : haute, moyenne, basse. Un exemple représentatif :

```
{
  "technicien": { "id": "t-01", "nom": "Marie Santini" },
  "interventions": [
    {
      "id": "itv-1001",
      "station": "Station Nord",
      "domaine": "Vignoble Patrimoine",
      "latitude": 42.703, "longitude": 9.347,
      "priorite": "haute",
      "statut": "planifiee",
      "datePrevue": "2026-06-15",
      "description": "Remplacement du capteur d'humidite defectueux.",
      "historique": [
        { "date": "2026-06-10", "action": "Intervention creee" }
      ]
    }
  ]
}
```

Url de lecture du JSON : utrera.ludovic.aflokkat-projet.fr/getInterventions.json